

My recommendation

Choose your second idea—Inspector-led commissioning, followed by Sentry-led monitoring—but change it from “two weeks, then forever” to an adaptive, per-host lifecycle.

The improved design is:

Sentry runs from day one. New hosts receive elevated Inspector coverage. Once their baseline is sufficiently supported and stable, Sentry becomes the primary monitor. Anomalies, uncertainty, drift, and random audits return their recent history to Inspector.

This is the better architectural starting point for your project, **not yet an experimentally proven winner**. You should compare it with Sentry-first routing using the same dataset and CPU budget.

Also, your biggest challenge is currently **not model size or architecture**:

- Your cascade can reproduce Inspector decisions well on LANL.
- Your Inspector’s actual within-host detection performance on LANL is approximately chance.
- Therefore, **improving routing alone cannot solve the detection problem**.

I’ll cover:

1. How to implement networking capture and the dataset.
 2. How to implement point **7A: rolling recalibration**.
 3. The Inspector–Sentry architecture I would use.
 4. Improvements worth prioritizing.
-

1. Implementing section 6: networking capture and dataset creation

1.1 First, correct three assumptions in the document

A. Snaplen 512 is not a reliable complete fix

Increasing snaplen from 160 to 512 helps, but **512 bytes can still truncate TLS ClientHello packets**. Modern ClientHellos can be substantially larger.

For your research capture, use **full-packet capture**, subject to authorization, storage, and privacy controls. Extract metadata afterward.

For example, using Wireshark’s `dumpcap`:

```
# List interfaces
dumpcap -D

# Example: full-packet capture, hourly rotation, bounded ring buffer
dumpcap -i <interface-number> -s 0 `
-b duration:3600 -b files:48 `
-w D:\capture\netsentinel.pcapng
```

This example retains approximately 48 hourly files, **not a guaranteed storage size**. Archive files before overwrite and monitor disk usage.

Full capture does not mean your detector must inspect application content. However, describe your approach accurately:

"No application-payload content inspection or TLS interception; detection uses flow and available protocol metadata."

An absolute "no DPI" claim can be confusing when you parse DNS and TLS handshakes.

B. QUIC does not make all hostname extraction impossible

QUIC Initial protection can be decoded by compatible passive analyzers for supported versions. Successful extraction still depends on complete capture, reassembly, analyzer support, and whether ECH hides the actual hostname.

Therefore:

- Disabling QUIC is a **controlled-lab simplification**.
- It is **not a production solution for unmanaged devices**.
- Keep a second dataset condition with normal QUIC/DoH behavior.

C. Queries for DoH resolver domains do not quantify hidden DNS traffic

A query for `chrome.cloudflare-dns.com` suggests resolver-related activity. It does **not by itself establish that 55% of all DNS activity is hidden inside DoH**.

Report separately:

- Visible conventional DNS.
- Connections to known resolver services.
- Hostnames recovered through TLS or QUIC.
- Connections whose names remain unknown.

Measure observability; do not infer it from one domain counter.

1.2 Put the capture sensor in the right place

A laptop on a switched LAN does **not automatically see everyone else's unicast traffic**.

Use one of these:

Capture approach	Suitable use
Capture on each consenting machine	Fastest prototype
Switch SPAN/mirror port	Central multi-host observation
Network TAP	More dependable passive collection
Gateway capture	Internet-bound traffic, subject to NAT visibility

For your 2–3 developer team, I would start with:

Per-machine capture on a few consenting machines, then merge normalized metadata using synchronized timestamps and stable host IDs.

If you already have a managed switch with SPAN, central capture is preferable.

Important requirements:

- Capture before NAT when possible.
- Synchronize clocks.
- Record DHCP/IP changes.
- Avoid counting the same flow twice if you combine sensor locations.
- Do not assume “IP address = permanent host identity.”

Clarify the data-diode deployment

These are different situations:

1. Bidirectional traffic is observed inside the protected network, and telemetry is exported one-way through a diode.
2. The sensor can physically observe only one direction of the original traffic.

In situation 2, response bytes, DNS answers, and some connection features may be unavailable.

Your dataset and model must match the actual visibility. Missing reverse-direction data must be marked **missing**, not interpreted as zero traffic.

Also, a passive NIDS does not require an active network scan. Build inventory from DHCP, ARP, DNS, and flow records first. Any active inventory collection should be separately authorized, especially in OT environments.

1.3 Build two capture conditions

Do not collect only traffic from browsers with privacy features disabled.

Condition A: controlled visibility

On consenting lab machines:

- Disable browser DoH and QUIC if needed.
- Verify effective policies in `chrome://policy` or `edge://policy` .

- Restart the browser.
- Check whether OS-level or application-specific encrypted DNS is still active.
- Capture complete packets.

Purpose:

Test whether category sequences contain useful detection information when naming metadata is available.

Condition B: realistic visibility

Leave normal browser/network settings enabled.

Purpose:

Measure how much performance degrades under QUIC, DoH, ECH, and missing names.

Keep the first broken 21 hours as a **separate degraded-observability test set**. Do not silently mix it into ordinary benign training data.

1.4 Add a capture-quality gate before collecting for days

Run a 30–60 minute pilot first.

Generate a small list of known, benign browsing activities on the test machines, then verify what your sensor actually sees.

Create a proposed script:

v2/scripts/capture_quality.py

It should report:

Metric	Why it matters
Truncated packet percentage	Identifies snaplen problems
Capture drops / Zeek capture-loss indicators	Identifies missing traffic
TLS handshake parsing success	Tests the naming path
DNS-name association coverage	Tests fallback naming
QUIC share and parse coverage	Measures realistic visibility
Named external-flow coverage	Measures resolver effectiveness
Unknown category share, by host	Detects blind spots

Metric	Why it matters
Missing feature rates	Prevents invalid model inputs
Hosts and active hours observed	Checks dataset coverage

Do not use 77% resolver coverage as a universal pass mark. That is a comparison point from another dataset.

Define the denominator explicitly, for example:

```
named_external_flow_coverage =
    external flows with a supported hostname association
    / all external flows
```

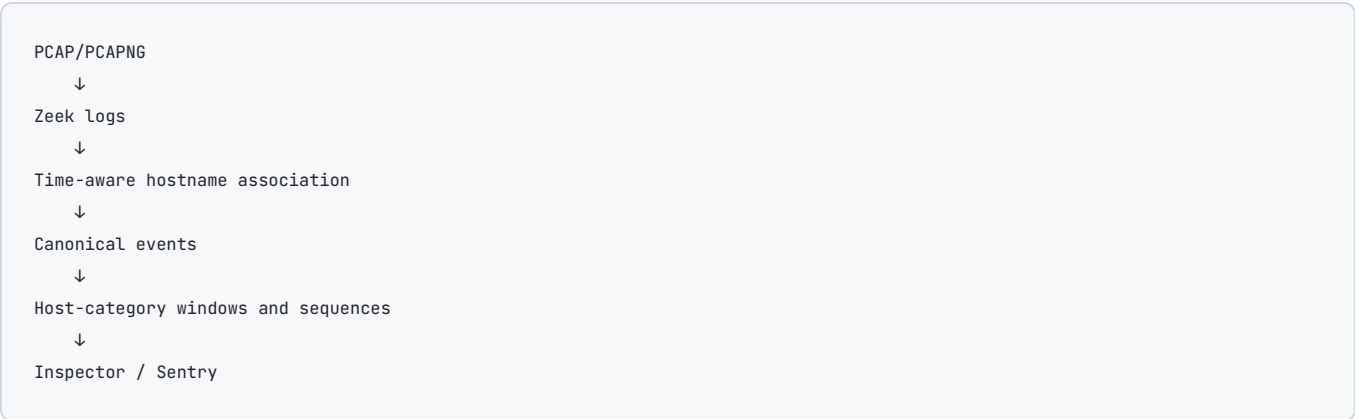
Also report **byte-weighted coverage**. Naming many tiny flows does not necessarily mean you understand most traffic.

1.5 Convert PCAPs into a canonical event dataset

Use your existing `zeekify.sh`, but test:

- Whether it accepts PCAPNG.
- Which Zeek version it uses.
- Whether QUIC parsing requires additional packages.
- Whether processing rotating files independently loses connections spanning file boundaries.

The pipeline should be:



Recommended event fields:

```
timestamp
sensor_id
host_id
connection_uid
destination_id
destination_port
transport
bytes_out
bytes_in
duration
```

```
hostname
hostname_source
hostname_confidence
service_category
category_confidence
visibility_flags
capture_condition
schema_version
```

Keep labels separately:

```
event_id / connection_uid
scenario_id
chain_id
scenario_start
scenario_end
label
label_source
```

Never include `label`, `scenario_id`, or injection markers in model features.

Make hostname association time-aware

Prefer direct connection-associated evidence such as SNI where available.

For DNS fallback:

- Associate answers with the originating host.
- Respect answer timing and TTL.
- Handle CNAMEs.
- Record ambiguous mappings.
- Do not use future DNS responses to label past connections.

Do **not** globally map a shared Cloudflare/CDN IP to one hostname. Shared hosting makes that unreliable.

1.6 Improve the taxonomy before training

Your categories currently mix **service identity** and **assumed purpose**:

- Dropbox appears as both `Cloud_Storage` and `Sync`.
- GitHub could mean browsing, software updates, or CI/CD.
- `Recon_API` sounds malicious even when someone legitimately checks their public IP.

A passive observer often cannot determine the exact purpose.

I would separate:

```
Service category:
  Code_Hosting
  Messaging
  Cloud_Storage
  Public_IP_Geolocation
  General_Web
```

```
Internal
Unknown
```

```
Behavior/context:
```

```
periodic
upload_heavy
interactive_like
new_for_host
common_for_peer_group
```

For the immediate deadline, keep your existing nine categories if changing them would require a major retrain—but document their ambiguity and add resolver confidence.

A category is evidence about a service, not proof of intent.

1.7 Create three dataset layers

Layer 1: observed background traffic

Collect different legitimate behaviors:

- Browsing.
- Development work.
- Repository access.
- Messaging.
- Cloud backup and synchronization.
- Software updates.
- CI jobs, if available.
- Idle periods.

Call it **observed background**, not guaranteed benign, unless you have supporting ground truth.

Record host roles, but do not use stable host identity as a shortcut for predicting attacks.

Layer 2: semi-synthetic chains

Your proposed Zeek-row injection is useful for a first experiment.

Inject controlled category sequences into copies of the background dataset, varying:

- Service choices.
- Inter-stage delays.
- Beacon jitter.
- Byte volumes.
- Missing stages.
- Interleaved background activity.
- Start times.
- Destination familiarity.

Keep the original captures immutable.

Maintain valid Zeek relationships: consistent timestamps, connection IDs, endpoints, and cross-log associations. Validate that the unchanged loader reconstructs the intended chain.

Layer 3: safe lab emulation

For stronger evidence, capture actual benign-purpose scripts that reproduce the network shape of a chain:

- Retrieve public-IP information.
- Download a harmless text file from an owned repository.
- Exchange dummy messages through a controlled test service.
- Upload a non-sensitive dummy file to an owned storage location.

Use only authorized accounts and infrastructure. No malware or real secrets are necessary.

Label these **emulated adversary-like chains**, not confirmed real-world intrusions.

Include difficult negatives

This is essential:

- CI pipeline: repository → messaging notification → artifact upload.
- Developer: public-IP lookup → GitHub → Discord → Drive.
- Scheduled backup with periodic traffic.
- Legitimate automation with low timing variance.

Otherwise, your model may learn:

“Automation is malicious” or “this four-category order is malicious.”

Neither is defensible.

1.8 Evaluate without leakage

For a 12-day pilot, an illustrative split is:

Days 1-6: model fitting
Days 7-8: calibration and configuration selection
Days 9-12: chronological test

Use gaps where necessary to prevent overlapping sequence windows crossing boundaries.

Also:

- Hold out some hosts if you have enough.
- Hold out scenario variants.
- Keep all copies of the same background segment in the same split.
- Fit scalars and baselines on training data only.
- Run multiple seeds, but remember that seeds do not replace independent captures.

A 12-day dataset cannot establish that a **14-day commissioning policy** works. Test shorter policies as a pilot or collect longer.

Report:

1. Within-host ROC-AUC and PR-AUC.
2. Chain/event detection recall.
3. False alerts per host-day.
4. Time to detection.
5. Inspector workload.
6. CPU time, memory, queue delay.
7. Performance by observability condition.
8. Teacher agreement separately from attack detection.

“Real background + injected chains” is a semi-synthetic benchmark, not real-attack validation.

2. Implementing point 7A: rolling recalibration

2.1 Separate the two thresholds

You need at least:

Threshold	Purpose
Sentry routing threshold	Selects windows for Inspector
Inspector alert threshold	Selects sufficiently unusual Inspector results

These are not interchangeable.

A 5% routing budget is a compute policy. A 1% Inspector exceedance target is a calibration choice. Neither is automatically an attack probability or guaranteed false-positive rate.

Also, your result shows **threshold miscalibration under changing traffic**, not necessarily a threshold that always “decays” in one direction.

2.2 Use guarded rolling quantiles, not automatic alert suppression

The basic idea is:

[$T_{\text{candidate}} = Q_{0.99}(S_{\text{reference}})$]

where the reference contains eligible, historical Inspector scores.

But this is dangerous if applied to all recent traffic:

An attack raises anomaly scores → the threshold rises → the attack becomes normal.

Therefore, do not implement “keep raising the threshold until only 1% is flagged.”

Recommended initial design

Create:

```
v2/netsentinel_v2/calibration.py
```

Maintain a reference buffer keyed by:

```
model_version  
feature_schema_version  
baseline_version  
host_group  
visibility_regime
```

Start with peer-group calibration. Use per-host thresholds only when the host has enough data.

A 99th percentile from a handful of highly correlated windows is unstable.

Eligible reference data

Use a combination of:

- Previously reviewed benign windows.
- Matured background windows with no known incident association.
- Random-audit windows, including those Sentry would not normally escalate.

Do not admit windows solely because their current score is low. That creates a self-selecting reference distribution.

Exclude or quarantine:

- Known incident intervals.
- Unresolved suspicious intervals.
- Capture failures.
- Taxonomy/schema changes.
- Windows from unsupported visibility conditions.

Even with these guards, some contamination is possible. Say so explicitly.

2.3 Fix the Inspector sampling bias

Once Sentry routes only unusual traffic, Inspector sees a **biased subset**.

If you recalibrate Inspector using only those scores, its threshold will be calibrated to already suspicious traffic rather than ordinary background.

For your first implementation:

Keep a separate random-audit stream of Sentry-normal windows and use it, with reviewed data, to maintain the Inspector reference.

If sample rates vary by host or group, record the sampling probabilities. Otherwise, busy or heavily inspected hosts may dominate calibration.

Random auditing reduces blind spots; it does not guarantee poisoning prevention.

2.4 Use a champion/challenger update

Instead of immediately replacing the threshold:

1. Calculate a candidate from past eligible scores.
2. Replay it against held-out historical reference data.
3. Compare it with the current threshold.
4. Check movement and data-quality limits.
5. Deploy or reject.
6. Log enough information to roll back.

Conceptual pseudocode:

```
def propose_threshold(reference, current, config):
    if reference.effective_samples < config.min_samples:
        return current, "insufficient_reference"

    if reference.capture_degraded:
        return current, "capture_degraded"

    if reference.unresolved_distribution_change:
        return current, "needs_review"

    candidate = quantile(reference.scores, 0.99)

    candidate = bound_threshold_change(
        current=current,
        candidate=candidate,
        scale=reference.robust_score_scale,
        max_step=config.max_step,
    )

    if not passes_shadow_checks(candidate, reference):
        return current, "shadow_check_failed"

    return candidate, "accepted"
```

Use a score-scale-aware movement bound rather than blindly allowing "10% threshold change": scores can be near zero or have arbitrary scales.

Do not cap actual incident alerts just because their rate exceeds the calibration target. Cap automatic adaptation, not evidence.

2.5 Keep baseline adaptation separate

There are three different operations:

1. Updating an alert threshold.
2. Updating the host’s normal-behavior baseline.
3. Retraining Inspector or Sentry.

Implement **threshold recalibration first** while keeping the scoring function fixed.

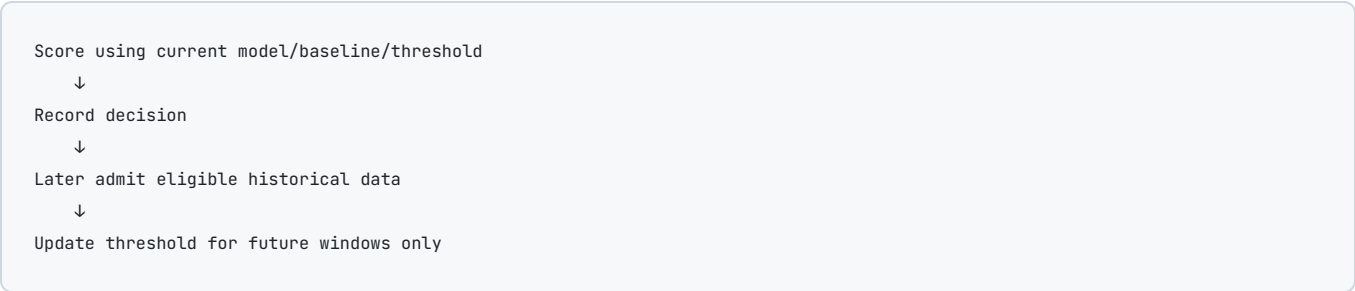
If the baseline or model changes, old scores may no longer be comparable. Recompute the reference under the new version or begin a versioned reference.

For baseline updates later:

- Maintain a frozen reference baseline.
- Build a candidate adaptive baseline.
- Promote it after checks or review.
- Freeze automatic updates during unresolved incidents.

2.6 Test recalibration chronologically

For each test window:



Never use the entire test period to calculate a threshold and then score that same period.

Compare:

Method	What it demonstrates
Fixed threshold	Existing behavior
Unguarded rolling percentile	Simple adaptation and its risks
Guarded rolling percentile	Your proposed improvement

Test under:

- Legitimate behavioral drift.
- Gradually introduced anomalous activity.
- Sudden incident bursts.

- DNS/SNI visibility loss.
- New hosts with little history.

Plot both **false alerts** and **attack recall**. A lower flag rate alone is not success.

3. Inspector–Sentry: which design is better?

3.1 Comparing your two options

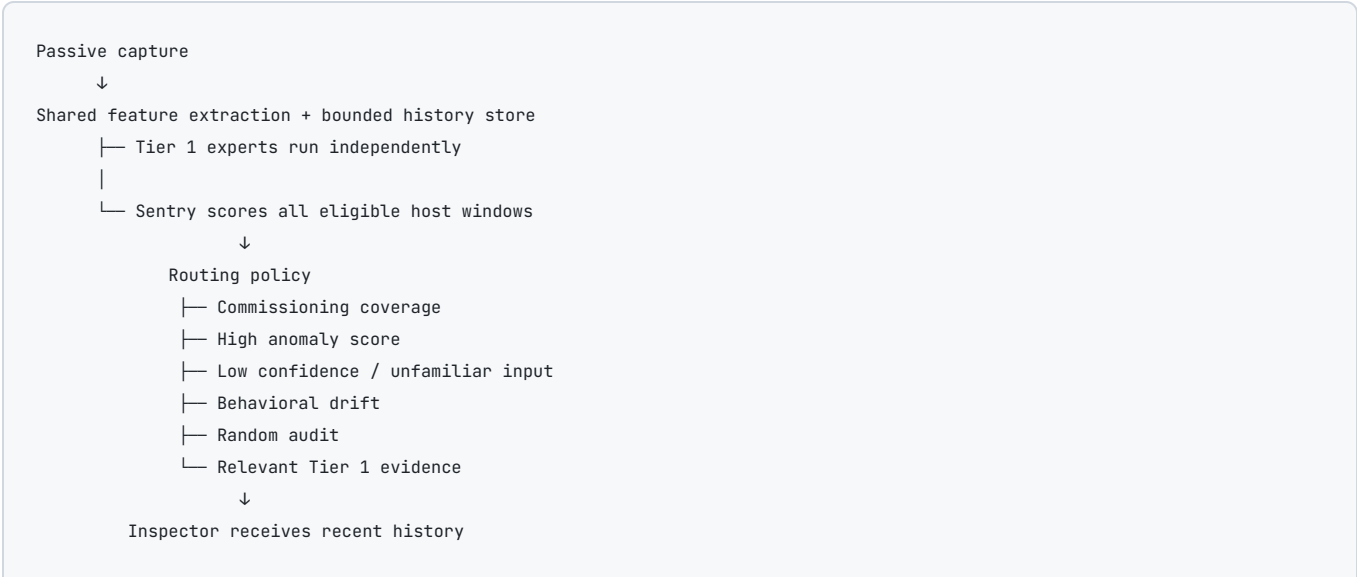
Design	Advantage	Main weakness
Sentry first from startup; escalate only anomalies	Cheapest	Cold-start blind spots and inherited Sentry errors
Inspector for all hosts for 14 days, then Sentry	Better initial coverage	Expensive; 14 days does not prove safety or completeness
Adaptive commissioning with Sentry always running	Balances initial coverage and long-term cost	Requires a small state machine and scheduling

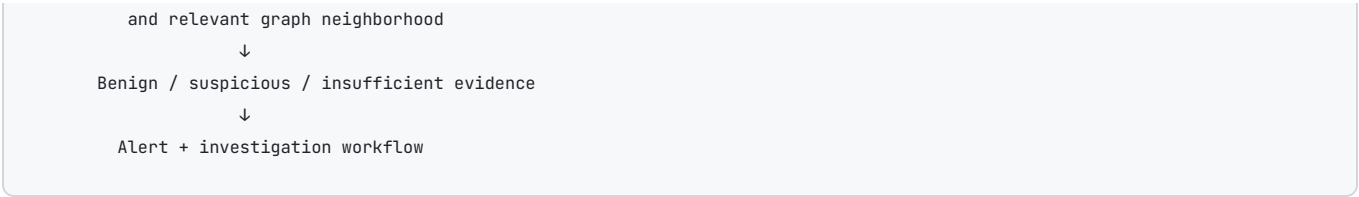
I choose the third—the improved version of your second idea.

For your small lab, you can run Inspector on every commissioning window **if measured CPU capacity allows it**. For a large network, use staggered commissioning and prioritized Inspector coverage.

Remember: route **feature windows and history**, not raw packets, to these models.

3.2 Recommended architecture





Inspector is a stronger assessor, **not an oracle**. An anomaly score alone does not confirm malicious intent.

3.3 Implement a per-host state machine

State	Behavior
COMMISSIONING	Sentry runs; elevated Inspector coverage
SENTRY_PRIMARY	Sentry handles routine monitoring
ESCALATED	Inspector examines current and historical context
INCIDENT_HOLD	Increased monitoring; baseline adaptation frozen
RECOMMISSIONING	Validate behavior following a legitimate change

Use a separate `visibility_status` field. A host can be in incident hold **and** have degraded capture.

Promotion should depend on evidence

Do not promote merely because a timer expired.

Check:

- Enough active observation.
- Coverage of relevant operating cycles.
- Sufficient reference samples.
- Acceptable naming/feature availability.
- Stable score distributions.
- No unresolved incident.
- Acceptable Sentry–Inspector disagreement.

Two weeks can be a reasonable initial observation target for some hosts, but:

- A quiet server may provide little evidence in two weeks.
- A monthly job may not appear at all.
- A development laptop may vary substantially day to day.

Replace “Inspector forever”

Use:

“Confirmed incidents remain under elevated monitoring until reviewed and explicitly released or

recommissioned."

A permanent hold for every suspicious result will eventually exhaust your CPU budget through false positives, legitimate changes, or deliberately noisy traffic.

3.4 Always send history with an escalation

Your hypothesis concerns chains. Sending Inspector only the last suspicious window can remove the very signal it needs.

Maintain:

- Short-term detailed event history.
- Medium-term host-category summaries.
- Longer-term transition statistics.

For a prototype, try 5-minute aggregation with a few hours of recent context, then tune this using measured chain durations. These are starting settings, not validated constants.

When Sentry escalates:

```
current window
+ preceding sequence
+ relevant graph neighborhood
+ visibility information
+ routing reason
```

Retain enough summaries to evaluate slower chains too.

3.5 Budget all Inspector work

A claimed 5% budget must account for:

- Commissioning.
- Anomaly escalations.
- Random audits.
- Incident holds.
- Repeated analysis.

Also, 5% of windows does not necessarily mean 5% of CPU: graph sizes and history lengths vary.

Measure:

Inspector CPU-seconds, queue delay, and event coverage—not only routed-window percentage.

Under overload, prioritize incident/high-risk work, deduplicate repeated escalations, and expose delayed coverage. Do not silently drop evidence or change alert thresholds to hide overload.

4. The improvements I would prioritize

Improvement 1: prove the sequence hypothesis with simpler baselines

Before adding another neural component, compare Inspector with:

- Category-frequency model.
- Transition/Markov model.
- Timing-and-volume-only model.
- Non-sequential anomaly detector.
- Same model with sequence order shuffled.

If shuffling the order barely affects performance, your experiment has not demonstrated that sequence order is the useful signal.

Also use traffic-volume-matched controls to address the busy-host confound you found on LANL.

Improvement 2: make “unknown” an explicit visibility condition

`Unknown_External` should not automatically mean suspicious.

Distinguish:

- Known hostname, uncategorized service.
- Hidden hostname.
- Parser failure.
- Truncated traffic.
- Raw-IP connection.
- Ambiguous CDN association.

Some require a data-quality warning, not a security alert.

Improvement 3: use neutral evidence explanations before an LLM

Generate a deterministic evidence object:

```
{
  "host_id": "host_17",
  "routing_reasons": ["rare_transition", "timing_change"],
  "sequence": ["Public_IP_Geolocation", "Code_Hosting", "Messaging", "Cloud_Storage"],
  "visibility": "partial",
```

```
"assessment": "suspicious",
"limitations": ["application purpose not visible"]
}
```

An optional LLM can summarize that evidence. It should not invent confirmation, calibrated confidence, or definitive ATT&CK mappings.

For protected environments, keep this local/offline or export only approved redacted information.

Improvement 4: fix Tier 1 input contracts before adding models

The port-scan schema mismatch must be fixed before judging detection quality. Feeding a 39-feature model the wrong 59-feature schema is not repaired by choosing a more scan-heavy test dataset.

Similarly, verify the VAE scaler mismatch with identical test vectors before assuming a version pin resolves its false-positive problem.

Add tests for:

- Feature names and order.
- Units.
- Directionality.
- Missing values.
- Training-versus-runtime transformations.
- Model/scaler version compatibility.

5. Practical implementation order for your team

Priority	Deliverable
1	Verified capture placement and 1-hour quality report
2	Continuous capture with controlled and realistic conditions
3	Canonical events, resolver provenance, immutable dataset manifest
4	Chain injection, hard negatives, chronological evaluation
5	Guarded recalibration with a random-audit reference
6	Inspector–Sentry state machine and history-based escalation
7	Actual CPU-budget and queue measurements

Priority	Deliverable
8	Evidence summaries and dashboard integration
9	Optional LLM narratives

Suggested division:

- **Developer 1:** capture, Zeek, resolver quality, dataset integrity.
- **Developer 2:** recalibration, runtime states, scheduling.
- **Developer 3, if available:** scenarios, evaluation, dashboard evidence.

Bottom line: Keep Inspector-led commissioning, but make it adaptive rather than a fixed two-week clearance. Run Sentry continuously, audit it randomly, and recalibrate carefully. Most importantly, build a dataset that can distinguish suspicious chains from legitimate automation—the current cascade results do not yet establish that distinction.

Yes—the main change is to replace snaplen 512 with full-packet capture (-s 0), then verify that the scheduled task actually uses the updated setting. Also fix the retention assumptions: your ring buffer limits storage, but does not guarantee that all 12 days survive.

I only have the commands you pasted, not the actual .ps1 contents, so the file changes below are search-and-edit instructions—not a verified patch.

1. Change the actual dumpcap command

Current command

```
dumpcap.exe -i <interface> -w D:\capture\ns.pcap `
-b duration:3600 -b files:<N> -b filesize:200000 -s 512
```

Recommended command for your authorized research capture

```
dumpcap.exe -i <interface> -w D:\capture\ns.pcapng `
-b duration:3600 -b files:<N> -b filesize:200000 -s 0
```

Changes:

Setting	Change	Reason
-s 512	-s 0	Capture complete packets; avoid truncating TLS ClientHellos
ns.pcap	ns.pcapng	Match dumpcap's usual default output format; the extension itself does not select the format

Setting	Change	Reason
Rotation settings	Keep initially	Reasonable, but monitor retention and disk usage

Correction to the previous instructions: `-s 512` is not "headers only, no payload" and is not guaranteed to capture TLS SNI. It captures the first 512 bytes of each packet, which can include application content while still truncating large handshake packets.

Full capture can contain sensitive data. Use it only on authorized machines/networks, restrict access, and retain extracted metadata rather than raw packets longer than necessary. No snaplen setting guarantees "metadata only."

2. Find where `512` is set in your scripts

Run:

```
Set-Location 'D:\1_sih26#2\netsentinel-main\v2'

$files = @(
    '.\start_capture.ps1',
    '.\capture_service.ps1',
    '.\install_capture_task.ps1'
)

Select-String -LiteralPath $files `
    -Pattern '512|snaplen|dumpcap|filesize|duration|MaxGB|\.pcap' `
    -Context 2,2
```

Back up the scripts before editing:

```
$stamp = Get-Date -Format 'yyyyMMdd-HH:mm:ss'

foreach ($file in $files) {
    Copy-Item -LiteralPath $file -Destination "${file}.${stamp}.bak"
}
```

In `start_capture.ps1` and `capture_service.ps1`

Look for whichever form your code uses:

```
-s 512
```

or:

```
"-s", "512"
```

or:

```
$SnapLen = 512
```

Change the corresponding setting to:

```
-s 0
```

or:

```
"-s", "0"
```

or:

```
$SnapLen = 0
```

Update comments too:

```
# Full-packet research capture.  
# May contain sensitive payload; apply access and retention controls.
```

Do not globally replace every occurrence of `512`—only the capture setting and related documentation.

In `install_capture_task.ps1`

Check whether it:

- Embeds the dumpcap arguments.
- Passes a snaplen parameter.
- Copies `capture_service.ps1` to another directory.
- Merely schedules the existing service script.

If it embeds `512`, change it. If it installs a copy, reinstall/update that copy using the installer. Editing your source file alone would not update an installed copy.

3. Check filename compatibility before switching to `.pcapng`

Your converter may search only for `*.pcap`.

Check:

```
Select-String -LiteralPath '.\zeekify.sh' `  
-Pattern 'pcap|find|glob' -Context 2,2
```

If it searches only for `.pcap`, update discovery to include both extensions. For example, a shell `find` expression could be:

```
find "$INPUT_DIR" -type f \( -name '*.pcap' -o -name '*.pcapng' \)
```

Adapt that to the script's existing loop; do not paste it blindly over the entire converter.

For the smallest immediate change, leave the existing filename unchanged and change only `-s 512` to `-s 0`. Then address format naming separately after a conversion test.

4. Restart capture—the running process will not change automatically

First inspect the currently running command:

```
Get-CimInstance Win32_Process -Filter "Name='dumppcap.exe'" |
    Select-Object ProcessId, ExecutablePath, CommandLine |
    Format-List
```

This tells you:

- Whether capture is still running.
- Which interface it uses.
- Whether it actually has `-s 512`.
- Where it writes output.

Then:

1. Stop capture using your script's documented stop mechanism. For an interactive dumppcap session, use **Ctrl+C**.
2. Update the scripts.
3. Reinstall/update the scheduled task if it uses copied files or embedded arguments.
4. Restart capture.
5. Run the process-inspection command again and confirm `-s 0`.

Your pasted instructions list:

```
.\install_capture_task.ps1 -Status
.\install_capture_task.ps1 -Uninstall
```

However, I cannot tell from the name whether `-Uninstall` also stops an already-running dumppcap process. Verify that before starting another capture.

Avoid two concurrent writers to the same output location.

Start the corrected capture in a separate folder

```
.\start_capture.ps1 -Interface 5 `
    -OutDir 'D:\capture-full' -MaxGB 40
```

For persistent capture, after confirming the previous task/process is stopped:

```
.\install_capture_task.ps1 -Interface 5 `
  -OutDir 'D:\capture-full' -MaxGB 40
```

Using a new folder preserves the distinction between truncated and corrected captures.

5. Fix the “12-day dataset” retention problem

These options:

```
-b duration:3600
-b filesize:200000
```

mean **rotate after one hour OR approximately 200 MB, whichever happens first.**

Therefore:

- Files may cover much less than one hour.
- `<N>` files do not necessarily represent `<N>` hours.
- Full capture can fill the ring faster than the previous truncated capture.
- `-MaxGB 40` is a storage budget, not a duration guarantee.

For your dataset:

Archive completed capture files outside the ring directory before they are overwritten.

Do not move the file currently being written. A later improvement is to make the service track completed files and archive them automatically.

Measure actual storage use for several representative hours:

```
$bytes = (
  Get-ChildItem 'D:\capture-full' -File |
  Measure-Object -Property Length -Sum
).Sum

'{0:N2} GiB currently stored' -f ($bytes / 1GB)
```

Use measured daily volume to size the archive. Keep hashes and a manifest of capture settings for reproducibility.

6. Keep your browser policies—for the controlled experiment only

Your QUIC/DoH policy commands can remain for the **controlled-visibility dataset**.

After applying them:

- Restart Chrome/Edge.
- Verify effective settings at `chrome://policy` or `edge://policy`.
- Check other applications and OS-level encrypted DNS separately.

But change the documentation from:

"Required for hostname visibility."

to:

"Controlled-lab settings to improve naming visibility; not a universal production requirement or guarantee."

QUIC can sometimes be passively parsed by compatible analyzers; ECH can still hide the real hostname. Browser policies do not control every application.

Later, collect a separately labeled dataset with normal settings enabled.

7. Validate with a short capture before leaving it running

After 10–15 minutes, inspect a **completed** capture file.

If Wireshark is installed at the usual location:

```
$tshark = 'C:\Program Files\Wireshark\tshark.exe'  
$pcap = 'D:\capture-full\<completed-file>.pcapng'
```

Check capture truncation

```
& $tshark -r $pcap `  
-Y 'frame.cap_len < frame.len' `  
-T fields -e frame.number |  
Measure-Object -Line
```

Ideally this reports zero truncated packets. It does **not** measure capture drops.

Check observable TLS SNI

```
& $tshark -r $pcap `  
-Y 'tls.handshake.extensions_server_name' `  
-T fields -e tls.handshake.extensions_server_name |  
Sort-Object -Unique
```

Check visible DNS queries

```
& $tshark -r $pcap `
-Y 'dns.flags.response == 0' `
-T fields -e dns.qry.name |
Sort-Object -Unique
```

Then run your existing Zeek conversion on the pilot and confirm the **Zeek loader**, not just Wireshark, recovers names correctly.

What to change now, in order

1. Inspect the running dumpcap command.
2. Change snaplen to in both interactive and persistent capture paths.
3. Restart and verify the effective command.
4. Capture into a new directory.
5. Run the short truncation/SNI/DNS/Zeek checks.
6. Archive completed files so the ring cannot erase your dataset.

You do **not** need to retrain models or rewrite the pipeline merely to make these capture corrections. First establish that the new capture produces complete, usable observations.

Generated 2026-09-10 · Astra Recommendations